when transferring data. The following listing shows a syntax comparison of two object literals related to Scruffy's favorite television show, *Futurama*.

Listing 4.1. Comparison of YAML literal with brace literal notation

YAML object literals

```
futurama =
  characters: [
    'Fry'
    'Leela'
    'Bender'
    'The Professor'
    'Scruffy'
  ]
  quotes: [
    'Good news everyone!'
    'Bite my shiny metal'
  ]
```

Both sides declare a variable with the name futurama that contains two properties named characters and quotes. The value of the characters property is an array containing five strings and the quotes property is an array with two strings.

Brace object literals

```
futurama = {
  characters: [
    'Fry',
    'Leela',
    'Bender',
    'The Professor',
    'Scruffy'
  ],
  quotes: [
    'Good news everyone!'
    'Bite my shiny metal'
  ]
}
```

That covers what objects look like; how about what they're used for? To start, the convenient literal notation for objects in CoffeeScript is useful for representing data such as key-value stores.

4.2. Key-value stores

A key-value store associates a set of keys with a set of values, each key being associated with one value. In CoffeeScript, objects are often used as key-value stores, similar to how a hash is used in Ruby, a hash table is used in Java, or a dictionary is used in Python.

Objects in CoffeeScript aren't hash tables in the strict sense of the word, but they are effective as key-value stores. In particular, the convenient object literal syntax and lack of ceremony make them very effective key-value stores. In this section you'll see how objects as key-value stores are used as data in a program and to name function arguments.

4.2.1. Data